Our Team name & team members

Team name : **ICT-EMLS**

Team Members:

No.	Name	Role	Affiliation
1	Junzhou Xu	Team member	Institute of Computing Technology, Chinese Academy of Sciences
2	Zijia An	Team member	
3	Ruiqi Liu	Team member	
4	Xi Zhang	Team member	
5	Runjie Shao	Team member	
6	Boyu Diao	Team member & Supervisor	

Short description about our solution

1. Sampling Balance Strategy

Existing Problems: We observed that the baseline method stores all seen samples in memory and randomly selects them to construct batches. This approach has two issues: 1) New samples in online learning are not guaranteed to be trained due to random sampling, leading to insufficient training on new tasks; 2) Recently learned samples may be repeatedly sampled, preventing the method from focusing on hard examples.

Solutions: To address these issues, we propose two solutions: 1) Prioritize adding untrained samples to each batch to ensure new samples in online learning are definitely learned; 2) Introduce a buffer to store recently learned samples, preventing repetitive training on the recent samples.

Detailed Code: In implementation, we divide each batch into two parts: unlearned samples and replay samples, according to solutions 1 and 2 respectively. For unlearned samples, we use NUM_ITER to determine the position of newly arrived samples in memory and sample them for the batch. For replay samples, we reserve part of the memory as a buffer zone, place recently learned samples into this buffer, and prohibit sampling from the buffer.

In my_utils.py

```
#TODO
if len(memory) <  total_batchsize:
     batch = random.sample(memory, k=min(len(memory), total_batchsize))
elif num_iterations<=1:
    if total_batchsize<=math.ceil(1 / num_iterations):
        new_num = total_batchsize
        old_num = 0
    else:
        new_num = math.ceil(1 / num_iterations)
        old_num =  max(0,total_batchsize - new_num)

    new_sample = memory[-new_num:]
    start_index = 0
    if len(memory)>2000:
        start_index = 600
    available_indices = list(range(start_index,len(memory) - new_num))
    old_chosen_indices = random.sample(available_indices, k=old_num)
    old_sample = [memory[i] for i in old_chosen_indices]
    batch = new_sample+old_sample

    remaining_old = [item for idx, item in enumerate(memory[:len(memory)-new_num]) \
                     if idx not in old_chosen_indices]
    # sort  new-old-remaing
    memory.clear()
    memory.extend(new_sample)
    memory.extend(old_sample)
    memory.extend(remaining_old)
else:
    new_num = 1
    new_sample = memory[-new_num:]
    old_num = total_batchsize -new_num
    if len(memory)>2000:
        start_index = 600
    available_indices = list(range(start_index,len(memory) - new_num))
    old_chosen_indices = random.sample(available_indices, k=old_num)
    old_sample = [memory[i] for i in old_chosen_indices]
    batch = new_sample+old_sample
    move_last_n_to_front_inplace(memory, new_num)
```

2. Finetuning and Learning Rate Strategy

Existing Problems: We observed that hyperparameter settings and fine-tuning targets affect performance. Incorrect LoRA parameters and frozen projection layers in the baseline reduced model plasticity, making it difficult to learn new tasks.

Solutions: We finetune LoRA parameters and unfroze vision-to-language projection layer (mm_projector) to ensure sufficient model plasticity for learning new tasks. However, we observed that introducing more learnable parameters caused forgetting issues. Therefore, we introduced a cosine annealing learning rate strategy to balance plasticity and stability. And we use a relatively large dropout rate（0.18） to prevent

overfitting. We expect to quickly learn new domain knowledge with high learning rates in early online learning stages, then reduce learning rates in later stages to mitigate forgetting caused by knowledge updates. Additionally, we found that reducing the learning rate of projection layers is equally crucial for maintaining the balance between stability and plasticity.

Detailed Code:

For finetuning strategy, We apply LoRA to all linear layers of the language model except for the embedding and lm_head layers, with the prameters:lora_r = 128; lora_alpha = 128; lora_dropout=0.18. Additionally, we set the vision-to-language projection layer (mm_projector) as trainable parameters.

```
def load_adapter(model, model_args, training_args, tokenizer, compute_dtype):
    if training_args.lora_enable:
        from peft import LoraConfig, get_peft_model
        training_args.lora_r = 128
        training_args.lora_alpha = 128
        training_args.lora_dropout=0.18
        lora_config = LoraConfig(
            r=training_args.lora_r,
            lora_alpha=training_args.lora_alpha,
            target_modules=find_all_linear_names(model),
            lora_dropout=training_args.lora_dropout,
            bias=training_args.lora_bias,
            task_type="CAUSAL_LM",
            # use_rslora = True,
            # init_lora_weights ="pissa",
            # use_dora = True
        )
        model = get_peft_model(model, lora_config)
        model.print_trainable_parameters()
        return model
```

During fine-tuning on the four downstream tasks, we freeze the mm_projector layer.

In my_trainer_downstream.py

```
# mm freeze
{
    "params": [
        p for n, p in opt_model.named_parameters() if (n in decay_parameters and n in projector_parameters and p.requires_grad)
    ],
    "weight_decay": self.args.weight_decay,
    # "lr": self.args.mm_projector_lr,
    "lr": 0.0,

},
{
    "params": [
        p for n, p in opt_model.named_parameters() if (n not in decay_parameters and n in projector_parameters and p.requires_grad)
    ],
    "weight_decay": 0.0,
    # "lr": self.args.mm_projector_lr,
    "lr": 0.0,
},
```

For learning rate stategy, we use an initial learning rate of LR = 3e-5 and MM_PROJECTOR_LR = 2e-5 , with WARMUP_RATIO = 0.04. The optimizer is adamw_torch, and the learning rate schedule employs cosine_with_min_lr, where the

minimum learning rate ratio (min_lr_rate) is set to 0.15.

```
LR=3e-5
MM_PROJECTOR_LR=2e-5
OPT_NAME="adamw_torch"
SCHED_NAME="cosine_with_min_lr"
WARMUP_RATIO=0.04
```

```
# this
scheduler_specific_kwargs={"min_lr_rate":0.15}
```

3. Parameter Constraint Strategy

Existing Problems: Existing approaches still lack fine-grained knowledge preservation strategies, posing risks of forgetting critical knowledge in long-term continual learning scenarios.

Solutions: We preserve important knowledge in a more fine-grained manner by evaluating parameter importance for old tasks and penalizing changes to critical parameters. We use the Fisher information matrix of old tasks to assess the importance of each parameter. However, online learning lacks task boundaries, and old task parameter importance changes as memory samples join training. Therefore, we use exponential moving averages to dynamically update a Fisher matrix that preserves old task parameter importance. The exponential moving average method not only preserves historical information but also helps forget incorrect parameter importance. Before optimization, we penalize changes to previous parameters based on the Fisher matrix, thereby reducing updates to important parameters.

Detailed Code: in my_trainer_upstream.py

```
def compute_loss(self, model, inputs, return_outputs=False):
    loss, outputs = super(CustomLLaVATrainer_Upstream, self).compute_loss(model, inputs, return_outputs=True)
    # add ewc
    if self.ewc_lambda > 0 and self.fisher_matrix:
        ewc_penalty = torch.tensor(0.0).to(self.args.device)
        for name, param in model.named_parameters():
            if param.requires_grad and name in self.fisher_matrix:
                fisher_value = self.fisher_matrix[name]
                optimal_param = self.optimal_params[name]
                ewc_penalty += (fisher_value * (param - optimal_param).pow(2)).sum()

        loss = loss + (self.ewc_lambda / 2) * ewc_penalty

    return (loss, outputs) if return_outputs else loss
```

```python
def _update_fisher_matrix(self, model: nn.Module):

    if not self.fisher_matrix:
        for name, param in model.named_parameters():
            if param.requires_grad and param.grad is not None:
                self.optimal_params[name] = param.data.clone().detach()
                self.fisher_matrix[name] = param.grad.data.clone().detach().pow(2)
    else:
        for name, param in model.named_parameters():
            if param.requires_grad and param.grad is not None:
                grad_data = param.grad.data.clone().detach()
                self.fisher_matrix[name] = (self.ewc_alpha * grad_data.pow(2)) + \
                                           ((1 - self.ewc_alpha) * self.fisher_matrix.get(name, torch.tensor(0.0).to(grad_data.device)))
                self.optimal_params[name] = param.data.clone().detach()
```

Any tips on running our code

We don't need any additional Python libraries; just follow the official instructions to run it.

(1)During evaluation, setting the batch_size to 1 significantly improves the accuracy of the generated outputs. We observed that when using batch_size = 2, in most cases only one of the two responses follows the instruction correctly, while the other is either gibberish or completely irrelevant. Therefore, we set batch_size = 1 to ensure consistent and reliable evaluation performance.

(2)We found that during fine-tuning on the four downstream tasks, the state are loaded sequentially rather than independently. Therefore, we made the following modifications （The modifications we made are marked with #this in the code, indicating two locations）：

In main.py

```python
    for downstream_idx in range(4):
        # You can customize how to initialize model weight for downstream tasks
        load_state_dict(model, state_dict, training_args)
        logger.info('model loading done')
        train_datalists = json.load(open(f'dataset/Downstream/train/dataset-{downstream_idx}.json', 'r'))
        random.shuffle(train_datalists)

        data_module = make_supervised_data_module(client_data=train_datalists, # sub_dataset
                                                  tokenizer=tokenizer,
                                                  data_args=copy.deepcopy(data_args))
        # ==========================================================================================
        # You are allowed to modify trainer only for optimizer initialization
        trainer = create_trainer_downstream(model,tokenizer,copied_args,data_module)
        results = trainer.train()
        # this
        # state_dict = get_state_dict(training_args, model)
        my_state_dict = get_state_dict(training_args, model)
        os.makedirs(f"{training_args.state_dir}", exist_ok=True)
        output_dir = os.path.join(training_args.state_dir, f"Downstream_{downstream_idx}_sc{training_args.scenario}.pth")
        if (training_args.local_rank == 0 or training_args.local_rank == -1):
            # this
            # torch.save(state_dict, output_dir)
            torch.save(my_state_dict, output_dir)
```

Since the competition rules and code did not specify that the above two parts cannot be modified, we submitted our results under these two settings.

(3) Additionally, we use a smaller learning rate during fine-tuning on downstream tasks to avoid disrupting the knowledge learned in upstream tasks. If this is not allowed, please comment out the two corresponding code sections.

"lr": 1e-5, #or If not allowed, you can comment it out.

In my_trainer_downstream.py

```
# less lr
optimizer_grouped_parameters = [
    {
        "params": [
            p for n, p in opt_model.named_parameters() if (n in decay_parameters and n not in projector_parameters and p.requires_grad)
        ],
        "weight_decay": self.args.weight_decay,
        "lr": 1e-5, #or If not allowed, you can comment it out.
    },
    {
        "params": [
            p for n, p in opt_model.named_parameters() if (n not in decay_parameters and n not in projector_parameters and p.requires_grad)
        ],
        "weight_decay": 0.0,
        "lr": 1e-5, #If not allowed, you can comment it out.
    },
```

Furthermore, experiments show that strong results can already be achieved with a relatively small NUM_ITER, such as 0.4. Increasing this value leads to further improvements in accuracy on both upstream and downstream tasks.